



TICKET REPETITION ANALYZER

Finding the Same Problem, Reported Differently

How AI-based duplicate detection cut through 11,700+ support tickets — and how a clustering accuracy fix surfaced hundreds more hidden repeats

Updated · Accuracy & Reliability Fixes Included

Project Review · STAR Summary · Prepared for CTO & Engineering Leadership

SITUATION

Same Issue, Different Words — and No Way to See It

- Support teams handle 11,000+ tickets across products and branches
- The same underlying problem is reported repeatedly in different wording
- Keyword search and manual tagging both miss these matches
- Result: no visibility into what's actually recurring, or how much effort is duplicated

"VPN not connecting"

=

"Unable to access VPN"

Same problem — invisible to keyword search

The cost: repeat effort is invisible — teams keep re-solving problems that were already fixed, with no data to prove it or prioritize it.

TASK

What We Set Out to Build

- Automatically detect tickets describing the same issue, regardless of wording
- Quantify how many tickets are duplicates, and how much of it is still unresolved
- Deliver it as a self-serve dashboard — readable by anyone, no engineering help needed
- Keep all ticket data on-premise — no sensitive data sent to an external service
- Make it work from day one on a real export — no long integration project required

ACTION

How It Works, Technically



Similarity threshold is tunable — lower it to catch more duplicates, raise it to avoid false matches.

Why it works: two tickets don't need matching keywords — only a similar meaning — so "VPN not connecting" and "unable to access VPN" land in the same group automatically.

ACTION

What We Delivered

- Upload-and-go: drag in a MongoDB export, no server setup
- Or submit a single ticket by hand from the dashboard — no MongoDB connection needed to test it
- 5 interactive dashboard views, switchable in one click
- Auto-recommended view — opens on whatever fits this data best, in plain English
- Runs fully on-premise: nothing leaves the machine
- ~11,700 tickets processed in about 1 minute

THE DASHBOARD, 5 VIEWS:

Overview

Trends

★ Recommended

Resolution insights

All clusters

New tickets

RESULT

The Impact, in Numbers

11,699

Tickets analyzed

994

Recurring issues found

37%

Repeat rate

3,329

Redundant tickets

224

Still open right now

- More than 1 in 3 tickets handled today is a repeat of a problem already reported
- 224 open tickets could potentially close the moment their original issue is fixed
- Team leads can now prioritize by actual repeat volume — not guesswork

A Hidden Accuracy Gap in Duplicate Detection

- The clustering engine required two tickets to share an **exact category label** before it would even check if they meant the same thing
- On real ticket data, category is inconsistent free text, not a clean taxonomy — e.g. "Easychit" vs. "EasyChit Client" for the same product
- Tickets worded almost identically (90%+ semantic similarity) were being kept apart purely because of a category string mismatch
- There was also no visibility into how long tickets actually stayed open, and testing a new ticket required a live MongoDB connection

The repeat rate the dashboard reported was an undercount — real duplicates were being missed silently, with no error or warning to flag it.

What We Set Out to Fix

- Make duplicate detection rely on meaning alone — not on inconsistent, free-text category labels
- Give team leads visibility into resolution time (average, fastest, slowest) and how often tickets reopen
- Let anyone submit a test ticket and see instantly whether it matches an existing issue — no MongoDB access required
- Fix the auto-recommended dashboard view so it always points at the most useful tab
- Do all of this without breaking the existing on-premise, no-cloud-data guarantee

The Fixes, Technically

- Removed the category guard rail from clustering (`src/cluster.py`) — matching is now purely cosine similarity between ticket embeddings
- Tracked ticket close time end-to-end (extractor, sample data, pipeline) to power new resolution-time metrics
- Built a self-serve "Submit a Ticket" form in the Flask app, wired to a new `add_manual_ticket()` pipeline path — embeds and re-clusters one ticket instantly and reports whether it matched an existing issue
- Corrected the auto-recommended-view logic so it points at the right dashboard tab instead of a mismatched one
- Pinned the embedding library (sentence-transformers) to a known-good version to stop a silent model regression

TOOLS & APPROACH

Python + Flask backend, sentence-transformers embedding model running fully on-premise, NumPy for similarity clustering, MongoDB/pymongo for ticket data, Anthropic Claude API for the natural-language `/ask` feature.

Validated directly against the live ~11,700-ticket export — comparing cluster counts before and after each fix, not synthetic samples alone.

What Changed, in Numbers

Metric	Before fix	After fix	Change
Tickets analyzed	11,692	11,699	+7
Recurring issues found	947	994	+47
Repeat rate	34%	37%	+3 pts
Redundant tickets identified	2,985	3,329	+344
Redundant & still open	191	224	+33
Avg. resolution time	not tracked	51.0 hrs	new
Reopened-ticket rate	not tracked	visible	new

344 previously-hidden duplicate tickets are now correctly grouped — found by removing one bad guard rail, not by relabeling a single ticket. Any team member can now self-test a ticket and get a same-page answer, with no MongoDB access or engineering help needed.